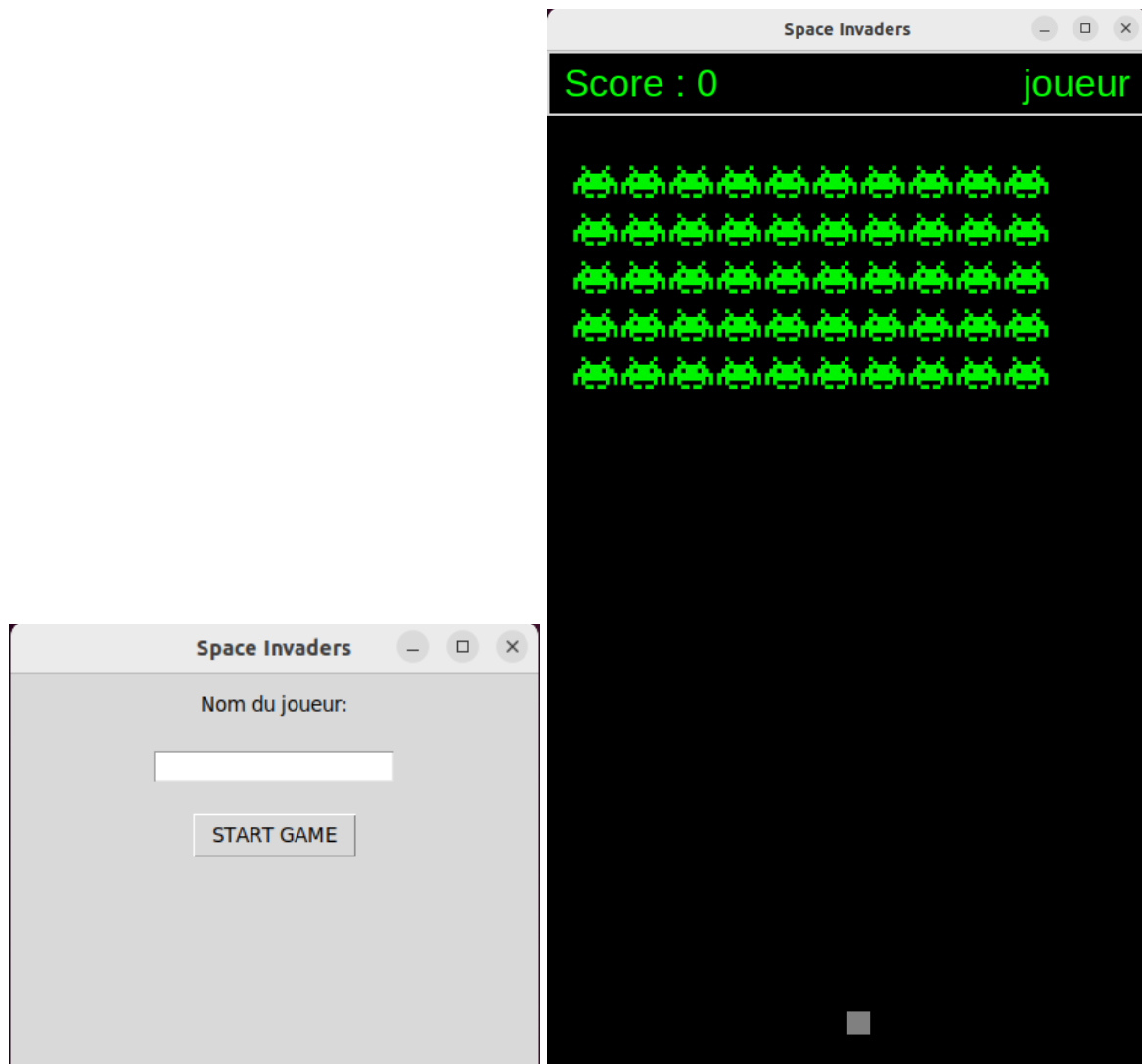


Documentation - Projet Space Invaders

Introduction

Le projet consiste à créer le jeu de Space Invader. Dans ce jeu, une flotte d'aliens continue d'avancer jusqu'à atteindre le défenseur situé en bas. Le joueur contrôle les mouvements de ce dernier, qui peut tirer des projectiles pour empêcher les aliens de l'atteindre en les éliminant.

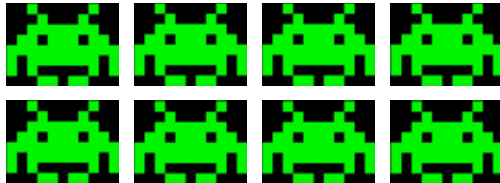
Voici les captures d'écran de la fenêtre d'accueil et de l'interface jeu :



1.1 - Le modèle



Alien



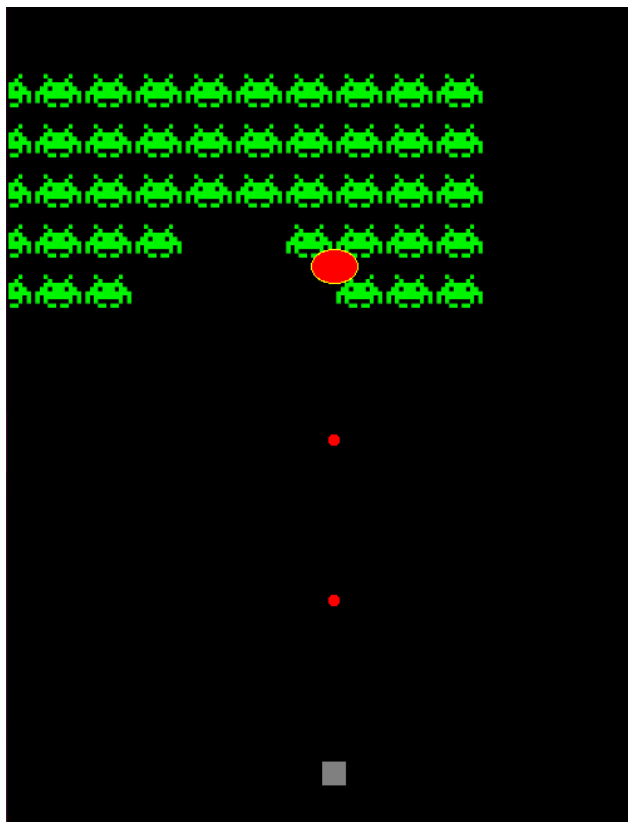
Fleet



Defender



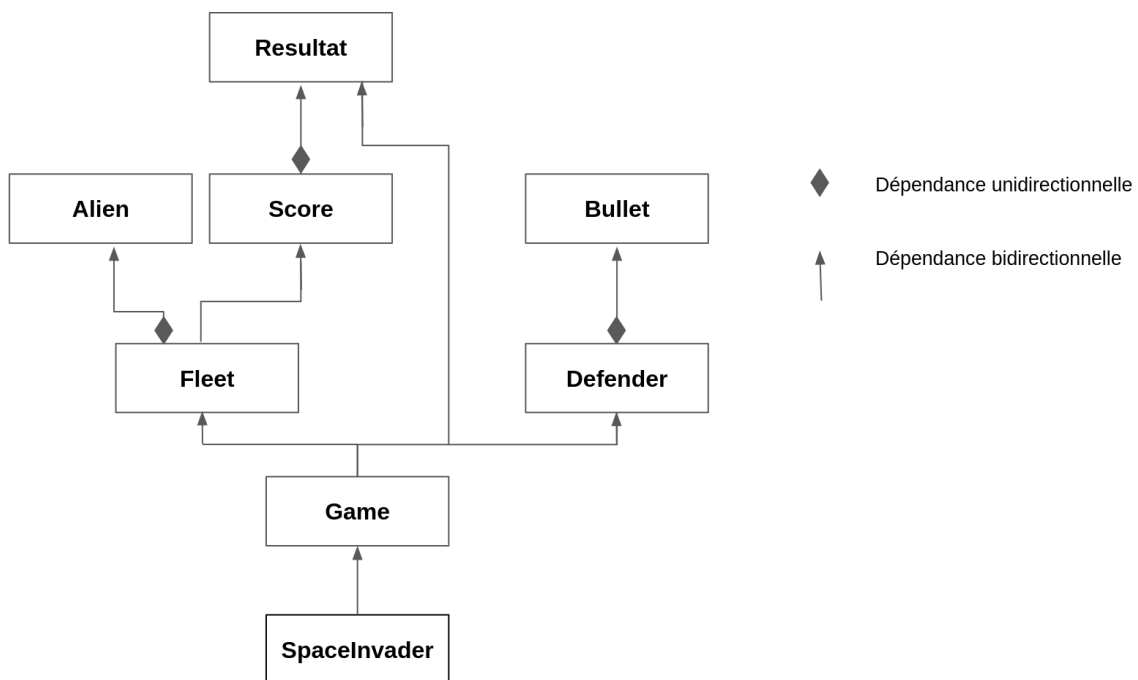
Bullet



Space Invaders

1.2 - Vue d'ensemble

Voici la vue d'ensemble qui explique la relation entre les différentes classes utilisées dans le projet :



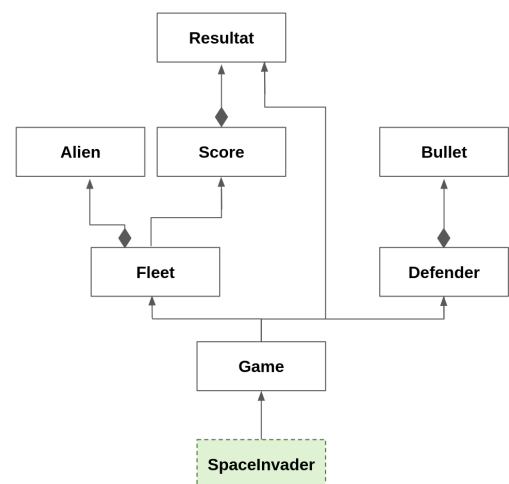
1.2.1 - SpaceInvader

```
class SpaceInvaders(object):
    """
    Main Game class
    """
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Space Invaders")
        self.player = None
        ...

    def play(self):
        self.root.mainloop()

    def recuperer_nom(self):
        ...
        self.startGame()

    def startGame(self):
        self.frame = tk.Frame(self.root,
                               self.frame.pack(side="top", fill
        self.game = Game(self.frame, self
        self.game.start_animation()
```



```
fen = SpaceInvaders()
fen.play()
```

C'est la classe principale qui initialise la fenêtre de jeu et l'interface d'accueil avec un champ pour saisir le nom du joueur et un bouton pour commencer le jeu "START GAME". Le nom du joueur est récupéré et, une fois récupéré, le joueur accède à l'interface jeu.

```
def __init__(self):
```

Dans cette définition d'instance, la fenêtre principale (self.root) est créée avec le titre "Space Invaders". Une variable self.player est utilisée pour stocker le nom du joueur, et un label (tk.Label) est ajouté pour indiquer que le nom du joueur doit être saisi. Un champ de saisie (tk.Entry) permet de récupérer le nom, et un bouton est placé pour démarrer le jeu. L'événement de clic sur ce bouton est lié à la méthode recuperer_nom().

```
def play(self):
```

Cette méthode lance la boucle principale de tkinter avec self.root.mainloop(). Cette boucle est essentielle pour maintenir l'interactivité de l'interface graphique, en traitant les événements et les saisies clavier du joueur pendant le jeu.

```
def recuperer_nom(self):
```

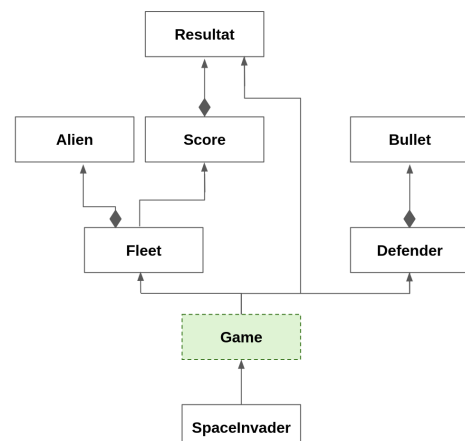
Cette méthode récupère le nom du joueur saisi dans le champ de texte (self.entree_nom). Si aucun nom n'est fourni, un nom par défaut, "joueur", est attribué. Une fois le nom récupéré, tous les éléments de l'interface d'accueil (le label, le champ de saisie et le bouton) sont supprimés pour faire place à l'interface du jeu. Enfin, la méthode appelle self.startGame() pour lancer le jeu.

```
def startGame(self):
```

Cette méthode initialise le jeu. Elle crée des frames pour afficher le nom du joueur et le score, ainsi qu'un frame pour le canvas. Ensuite, elle crée une instance de la classe Game. Enfin, elle démarre l'animation du jeu en appelant self.game.start_animation().

1.2.2 - Game

```
class Game(object):
    def __init__(self, frame, frame_labels,
player):
        self.frame = frame
        self.frame_labels = frame_labels
        self.gameOver = False
        self.fleet = Fleet(self)
        self.defender = Defender()
        self.height = 800
        self.width = 500
        self.player = player
        self.score = 0
        self.resultat = Resultat()
        self.listeScores = None
        self.canvas = tk.Canvas(self.frame,
width=self.width,
height=self.height, bg='black')
        self.canvas.pack(fill="both", expand=True)
```



```

        self.defender.install_in(self.canvas)
        self.fleet.install_in(self.canvas)
        self.canvas.focus_set()
        self.canvas.bind("<KeyPress>", self.defender.keypress)
        self.start_time = time.time()
        ...

    def update_score(self):
        ...
    def animation(self):
        ...
    def start_animation(self):
        ...
    def move_bullets(self):
        ...
    def move.aliens.fleet(self):
        ...
    def affiche_score(self):
        ...

```

Cette classe gère la logique du jeu : elle affiche le score du joueur, gère la boucle d'animation, et le déplacement des aliens et des tirs. Elle contient des méthodes pour gérer le score (`update_score()`), l'animation du jeu (`move_bullets()`, `move.aliens.fleet()`), et la gestion des collisions entre les projectiles et les aliens (`manage_touched.aliens.by()`).

```
def __init__(self, frame, frame_labels, player):
```

Cette méthode de définition d'instance prend plusieurs arguments : le frame créé dans la classe `SpacInvaders`, qui va contenir le canvas ; le `frame_labels`, qui contiendra les labels affichant le nom du joueur et son score lorsqu'il gagne aussi créé dans la classe `SpacInvaders` ; et enfin le nom du joueur, `player`, qui est stocké dans la variable d'instance (`self.player`).

Dans cette méthode, chaque instance possède les éléments suivants :

- un frame pour le canvas,
- un frame pour les labels,
- une variable booléenne `self.gameOver` qui indique l'état du jeu (en cours ou terminé),
- une instance de `Fleet` initialisée dans `self.fleet`, Une instance de `Defender` initialisée dans `self.defender`,
- les variables `self.height` et `self.width` définissant respectivement la hauteur et la largeur du canvas,
- une instance de `Resultat` dans `self.resultat`, destinée à stocker les scores dans un fichier ("lesScores.json"),
- une liste de scores `self.listeScore`,
- et enfin, le canvas lui-même, stocké dans `self.canvas`.

Lors de l'initialisation, la méthode lance l'installation de la flotte d'aliens (`self.fleet.install_in(self.canvas)`) ainsi que l'installation du défenseur (`self.defender.install_in(self.canvas)`). Elle active également le focus du clavier sur le canvas avec `self.canvas.focus_set()` et associe l'événement de pression de touche à la méthode `keypress` de la classe `Defender` via `self.canvas.bind("<KeyPress>", self.defender.keypress)`.

La variable `self.start_time` est initialisée avec la fonction `time()` du module `time`, ce qui permet d'enregistrer le moment précis du début du jeu, exprimé en secondes depuis le 1er janvier 1970 à 00:00. Enfin, un label est créé pour afficher le score, un autre pour afficher le nom du joueur, et les colonnes de la grille sont configurées via la méthode `grid_columnconfigure(0, weight=1)` pour s'assurer que les labels prennent toute la largeur disponible dans le `frame_labels`, en fonction de l'espace libre.

```
def update_score(self):
```

Cette méthode de classe permet de mettre à jour le score du joueur en fonction du temps écoulé depuis le début de la partie. Plus le temps passe, plus le score du joueur diminue, ce qui signifie que le joueur doit accomplir ses objectifs le plus rapidement possible pour obtenir un meilleur score. Le score est actualisé toutes les secondes (1 seconde = 1000 millisecondes).

```
def animation(self):
```

Tant que le jeu n'est pas terminé (`self.gameOver` est `False`), cette méthode est appelée pour animer le jeu. Elle met à jour la position des projectiles (`self.move_bullets()`), déplace la flotte d'aliens (`self.move.aliens_fleet()`) et met à jour le score du joueur (`self.update_score()`).

```
def start_animation(self):
```

Cette méthode sert principalement de "déclencheur" pour démarrer l'animation du jeu en appelant la méthode `animation()`.

```
def move_bullets(self):
```

Cette méthode déplace les projectiles tirés par le défenseur (le joueur) vers le haut. Si le jeu est fini, la méthode retourne immédiatement `None`, ce qui signifie que le mouvement des projectiles ne sera plus mis à jour et que la boucle s'arrête. Si un projectile sort du champ de vision (dépasse le haut de l'écran), il est supprimé. Les projectiles sont déplacés vers le haut de l'écran à chaque appel de `move_bullets()`. La méthode vérifie également les collisions entre les projectiles et les aliens (`self.fleet.manage_touched.aliens_by(self.canvas, self.defender)`), ce qui peut entraîner la destruction d'un alien et une mise à jour du score. Enfin, elle se relance périodiquement toutes les 10 millisecondes pour maintenir l'animation fluide (`self.canvas.after(100, self.move.aliens_fleet)`).

```
def move.aliens_fleet(self):
```

Cette méthode est appelée en boucle, et à chaque itération, elle déplace la flotte d'aliens via `self.fleet.move_in(self.canvas)` tant que le jeu n'est pas terminé.

```
def affiche_score(self):
```

Cette méthode est utilisée pour afficher la liste des scores (`self.listeScores`) sur le canvas. Elle s'assure que le canvas est réinitialisé (`self.canvas.delete("all")`) avant d'afficher soit les scores des joueurs, soit un message indiquant qu'aucun score n'est disponible. Elle crée un label pour chaque score dans la liste.

1.2.3 - Defender

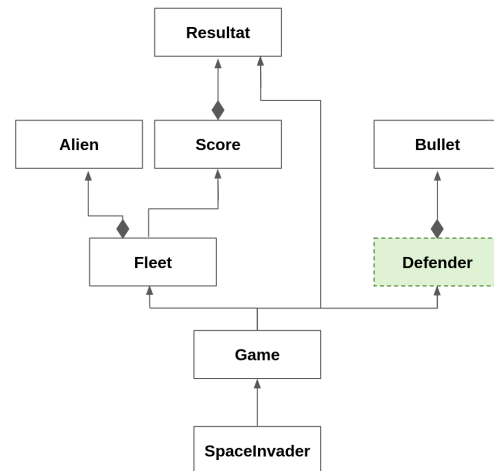
```
class Defender(object):
    def __init__(self):
        self.width = 20
        self.height = 20
        self.move_delta = 20
        self.id = None
        self.max_fired_bullets = 8
        self.fired_bullets = []

    def install_in(self, canvas):
        x, y = 250, 750
        self.id =
        canvas.create_rectangle(x, y, x +
        self.width, y + self.height,
        fill='gray', tags='defender')

    def move_in(self, canvas, dx):
        canvas.move(self.id, dx, 0)

    def fire(self, canvas):
        if len(self.fired_bullets) < self.max_fired_bullets:
            bullet = Bullet(self)
            bullet.install_in(canvas)
            self.fired_bullets.append(bullet)

    def keypress(self, event):
        ...
```



Cette classe représente le joueur (le défenseur). Il peut se déplacer à gauche et à droite avec les touches fléchées (droite et gauche) et tirer des projectiles avec la barre d'espace. La classe gère également les limites du mouvement pour éviter que le défenseur ne sorte du cadre du jeu.

```
def __init__(self):
```

Cette méthode de définition d'instance initialise chaque objet avec les éléments suivants:

- une largeur (self.width),
- une hauteur (self.height),
- une distance de déplacement (self.move_delta),
- un identifiant (self.id),
- une limite pour le nombre de projectiles tirés (self.max_fired_bullets),
- et une liste pour garder une trace des projectiles tirés (self.fired_bullets).

```
def install_in(self, canvas):
```

Cette méthode sert à placer le défenseur sur le *canvas* en créant un rectangle à des coordonnées spécifiques correspondant au milieu du *canvas*, en lui attribuant des dimensions et une couleur grise.

```
def move_in(self, canvas, dx):
```

Cette méthode permet de déplacer le defender sur le canvas (`canvas.move(self.id, dx, 0)`) en réponse aux actions du joueur grâce à deux paramètres: le canvas et dx qui correspond au déplacement horizontal de l'objet (sur l'axe des x).

```
def fire(self, canvas):
```

Cette méthode gère la création et l'affichage de projectiles sur le *canvas* dans le jeu. Elle commence par vérifier si le nombre de projectiles tirés est inférieur au nombre maximal autorisé (`self.max_fired_bullets`). Si c'est le cas, un nouveau projectile peut être tiré.

À chaque fois, elle s'assure que le nombre de projectiles ne dépasse pas un maximum défini, puis crée un projectile: un nouvel objet *Bullet* est créé et ajouté au canvas, puis affiché sur la canvas (`bullet.install_in(canvas)`). Le projectile est ajouté à la liste `self.fired_bullets`, permettant de garder une trace de tous les projectiles actuellement actifs pour faciliter la gestion des collisions et des animations..

```
def keypress(self, event):
```

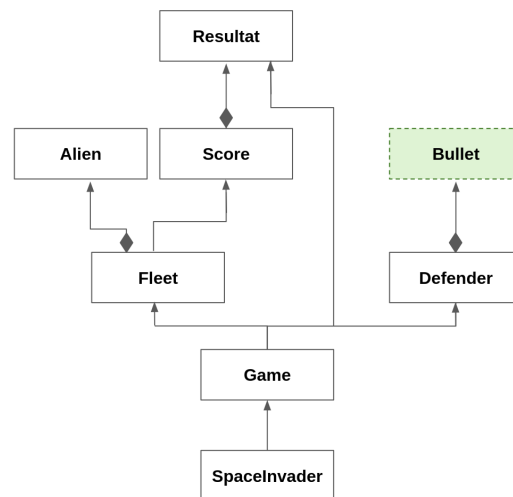
Cette méthode est liée à un événement de type "touche pressée" dans l'interface graphique (ici, dans le canvas tkinter avec `canvas = event.widget`), et elle prend en compte les touches directionnelles (gauche (`event.keysym == 'Left'`) et droite (`event.keysym == 'Right'`)) pour déplacer le défenseur ainsi que la touche espace (`event.keysym == 'space'`) pour tirer des projectiles (`self.fire(canvas)`).

1.2.4 - Bullet

```
class Bullet(object):
    def __init__(self, shooter):
        self.radius = 5
        self.color = "red"
        self.speed = 8
        self.id = None
        self.shooter = shooter

    def install_in(self, canvas):
        x1, y1, x2, y2 =
canvas.coords(self.shooter.id)
        cx = (x1 + x2) / 2
        cy = y1
        self.id =
canvas.create_oval(cx - self.radius,
cy - self.radius, cx + self.radius, cy + self.radius, fill=self.color)

    def move_in(self, canvas):
        canvas.move(self.id, 0, -self.speed)
```



Représente un projectile tiré par le joueur (le défenseur) qui se déplace vers le haut du canvas. Elle est appelée dans la classe *Defender* à chaque fois que le joueur enfonce la touche espace.

```
def __init__(self, shooter):
```


Cette méthode de définition d'instance prend en argument le shooter qui correspond au défenseur qui a tiré ce projectile ce qui lui permet de repérer sa position sur la canvas et ajuster la position des projectiles tirés.

Voici les différentes variables qui sont définies dans le constructeur pour chaque instance de la classe:

- le rayon du projectile (self.radius),
- la couleur du projectile (self.color),
- la vitesse du projectile (self.speed),
- une variable pour stocker l'identifiant de l'objet du projectile sur le canvas (self.id),
- et une variable qui stocke une référence à l'objet qui a tiré ce projectile (self.shooter).

```
def install_in(self, canvas):
```

Cette méthode crée et place un projectile (représenté par un cercle de couleur rouge) sur la *canvas* à une position déterminée par l'objet qui l'a tiré. Elle assure que le projectile apparaît correctement au-dessus du tireur et au centre horizontal de celui-ci.

```
def move_in(self, canvas):
```

Cette méthode permet de déplacer le projectile de manière verticale vers le haut sur le canvas. Le déplacement est effectué à chaque appel de la méthode, et la vitesse du projectile est contrôlée par la variable self.speed.

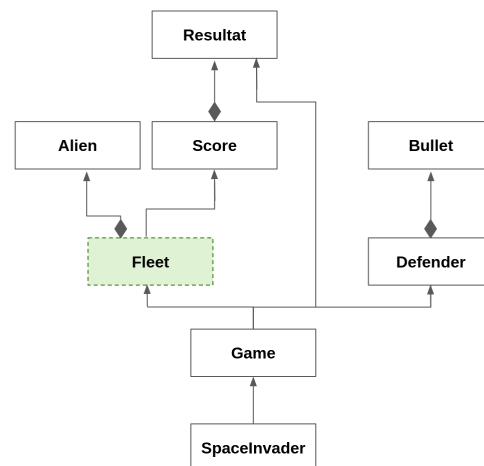
1.2.5 - Fleet

```
class Fleet(object):
    def __init__(self, game):
        self.game = game
        self.aliens_lines = 5
        self.aliens_columns = 10
        self.aliens_inner_gap = 20
        self.alien_x_delta = 20
        self.alien_y_delta = 15
        fleet_size = self.aliens_lines *
self.aliens_columns
        self.aliens_fleet = [None] *
fleet_size
        self.pim =
tk.PhotoImage(file='alien.gif')
        self.pim = self.pim.subsample(2,
2)
        self.gameOver = self.game.gameOver

    def install_in(self, canvas):
        ...

    def move_in(self, canvas):
        ...

    def manage_touched Aliens_by(self, canvas, defender):
        ...
```



Cette classe représente la flotte d'aliens. Elle gère la création, l'affichage, le mouvement, et les interactions des aliens avec le joueur comme la détection des collisions avec les projectiles.

```
def __init__(self, game):
```

Cette méthode de définition d'instance a pour argument l'objet game créé dans la classe SpaceInvaders qui va permettre d'accéder à la variable self.gameOver pour gérer quand arrêter le jeu. Elle permet d'initialiser les paramètres nécessaires pour gérer la flotte d'aliens et l'image associée suivants:

- l'attribut self.game qui sauvegarde une référence à l'instance de la classe Game récupérée en paramètre (game),
- une variable self.aliens_lines qui définit le nombre de lignes d'aliens dans la flotte,
- une variable self.aliens_columns qui définit le nombre de colonnes d'aliens,
- un attribut self.aliens_inner_gap qui détermine l'écart entre chaque alien dans la flotte,
- une variable self.alien_x_delta qui définit le déplacement horizontal des aliens lorsqu'ils se déplacent sur le canvas,
- une variable self.alien_y_delta qui définit le déplacement vertical,
- fleet_size qui calcule le nombre total d'aliens dans la flotte,
- une liste self.aliens_fleet avec le nombre d'éléments égal à fleet_size initialisé à None,
- self.pim une variable qui permet le chargement et redimensionnement de l'image d'un alien,
- et self.gameOver booléen qui sauvegarde une référence à l'attribut gameOver de l'instance de la classe Game.

```
def move_in(self, canvas):
```

Cette méthode est responsable de l'installation de la flotte d'aliens sur le canvas graphique du jeu qui récupéré en paramètre. Elle commence par définir les positions de départ et l'espacement entre les aliens. Ensuite, elle crée une matrice d'aliens en parcourant chaque ligne et chaque colonne, en plaçant un alien à chaque intersection de la matrice. Pour chaque alien, elle utilise la méthode install_in de la classe Alien pour l'ajouter au canvas à la position calculée, avec l'image correspondante et un tag "alien" pour l'identifier. Les positions des aliens sont ajustées horizontalement (en fonction de l'espacement) et verticalement (en descendant d'une ligne après chaque série de colonnes). Chaque alien est également ajouté à la liste self.aliens_fleet pour être suivi et manipulé ultérieurement dans le jeu

```
def manage_touched_alien_by(self, canvas, defender):
```

Cette méthode a pour paramètre le canvas et le défenseur. Elle déplace les aliens sur le canvas, détecte si la flotte a touché un bord de l'écran et inverse leur direction si nécessaire. Elle déplace les aliens horizontalement et, si un bord est touché, déplace la flotte verticalement d'une ligne.

Elle permet de supprimer l'alien s'il est touché (mort) du canvas en retirant l'identifiant de l'alien de la liste self.aliens_fleet (self.aliens_fleet.remove(alien)) et de supprimer les projectiles qui ont touché des aliens du canvas et de la liste fired_bullets (defender.fired_bullets.remove(bullet)).

Lorsque tous les aliens sont éliminés, elle affiche un message de victoire, marque la fin du jeu, et met à jour l'état global du jeu. Ensuite, elle enregistre le score du joueur dans un fichier JSON (lesScores.json), le charge à nouveau, et affiche les scores mis à jour à l'écran. Cela permet de sauvegarder et d'afficher les résultats du joueur à la fin de la partie.

Si un alien atteint la défense (le joueur), le jeu se termine et un message de "Game Over" est affiché. Cette méthode permet donc de gérer à la fois les déplacements des aliens et la logique associée au déroulement du jeu ainsi que la victoire ou défaite du joueur.

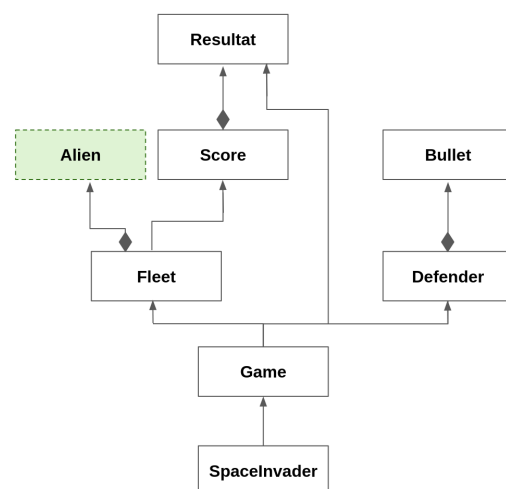
1.2.6 - Alien

```
class Alien(object):
    def __init__(self):
        self.id = None
        self.alive = True
        self.explosion_image = None

    def touched_by(self, canvas,
projectile):
        ...

    def install_in(self, canvas, x, y,
img=None, tag=None):
        self.id = canvas.create_image(x,
y, image=img, tags=tag)

    def move_in(self, canvas, dx, dy):
        canvas.move(self.id, dx, dy)
```



Cette classe représente un alien individuel. Elle gère son déplacement et sa collision avec les projectiles du défenseur. En effet, lorsqu'un alien est touché par un projectile, une explosion est créée à sa place avant qu'il ne soit supprimé.

```
def __init__(self):
```

Cette méthode de définition d'instance initialise les attributs suivants au moment de la création de l'instance de l'alien:

- self.id : un identifiant unique pour l'alien (initialisé à None),
- self.alive : un booléen pour savoir si l'alien est vivant ou non (initialisé à True),
- self.explosion_image : une variable pour gérer l'explosion de l'alien (initialisé à None).

```
def touched_by(self, canvas, projectile):
```

Cette méthode détecte si un alien est touché par un projectile dans un jeu. Elle prend en paramètre le canvas et le projectile pour gérer sa collision avec les aliens. Elle vérifie d'abord si l'alien est déjà mort (en contrôlant l'attribut self.alive), et si c'est le cas, elle retourne False pour indiquer qu'il n'y a pas de collision. Ensuite, elle compare les coordonnées de l'alien et du projectile pour déterminer s'il y a une collision. Si une collision

est détectée, l'alien est marqué comme mort (attribut `self.alive` est mit à `False`), et il est supprimé du canvas.

Un effet visuel d'explosion est ensuite créé à la position de l'alien avec `self.explosion_image`, et cet effet est supprimé après un court délai (100 millisecondes). La méthode retourne `True` pour indiquer qu'une collision a eu lieu, sinon elle retourne `False`.

```
def install_in(self, canvas, x, y, img=None, tag=None):
```

Cette méthode permet d'afficher un alien sur le canvas. Elle prend en entrée les coordonnées `x, y` pour positionner l'objet, une image `img` à afficher et un tag pour identifier l'objet. Elle utilise la méthode `canvas.create_image` pour dessiner l'image à l'emplacement spécifié et lui associe un id unique, stocké dans `self.id`, ce qui permet de manipuler l'image par la suite.

```
def move_in(self, canvas, dx, dy):
```

Cette méthode permet de déplacer un alien sur le canvas en utilisant les paramètres `dx` et `dy`, qui représentent les déplacements horizontaux et verticaux. Elle appelle la méthode `canvas.move`, qui prend l'identifiant de l'objet à déplacer (`self.id`) ainsi que les valeurs de déplacement `dx` et `dy`. Cela permet de modifier la position de l'objet en le déplaçant sur l'axe des `x` (horizontalement) et sur l'axe des `y` (verticalement).

1.2.7 - Score

```
class Score(object):
    def __init__(self, nom,
points):
        self.joueur = nom
        self.nbPoints = points

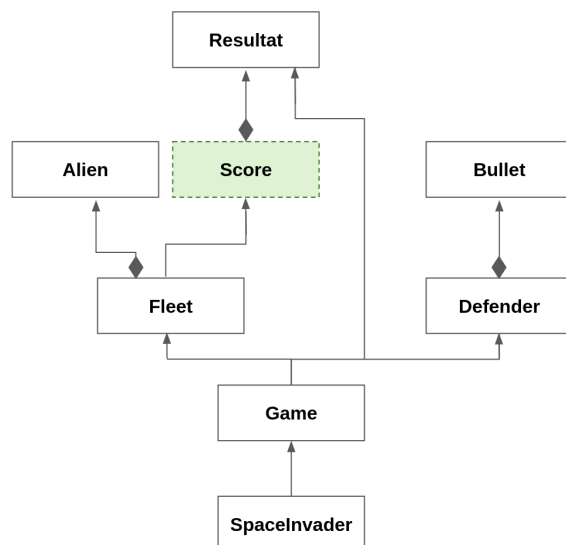
    def get_joueur(self):
        return self.joueur

    def get_nbPoints(self):
        return self.nbPoints

    def set_joueur(self, joueur):
        self.joueur = joueur

    def set_nbPoints(self, pts):
        self.nbPoints = pts

    def __str__(self):
        return self.joueur + " : "+ str(self.nbPoints)
```



Cette classe représente un score avec un nom de joueur et un nombre de points. Chaque score est ajouté à la classe `Resultat`.

```
def __init__(self, nom, points):
```

La méthode de définition d'instance prend deux paramètres :

- nom : le nom du joueur.
- points : le nombre de points du joueur, représentant le temps écoulé en secondes depuis le début du jeu.

Ces valeurs sont ensuite affectées aux attributs d'instance `self.joueur` et `self.nbPoints`.

```
def get_joueur(self):
```

Cette méthode permet d'accéder à l'attribut privé de l'objet (`self.joueur`) et retourne le nom du joueur.

```
def get_nbPoints(self):
```

Cette méthode permet d'accéder à l'attribut privé de l'objet (`self.nbPoints`) et retourne le nombre de points du joueur.

```
def set_joueur(self, joueur):
```

Cette méthode permet de modifier le nom du joueur en le remplaçant par le nom récupéré en paramètre `joueur`.

```
def set_nbPoints(self, pts):
```

Cette méthode permet de modifier les points du joueur en les remplaçant par les nouveaux points `pts` récupérés en paramètre.

```
def __str__(self):
```

Cette méthode spéciale définit la représentation en chaîne de caractères d'un objet. Elle permet d'afficher facilement le score du joueur dans une forme lisible: `nom_joueur : score`.

1.2.8 - Resultat

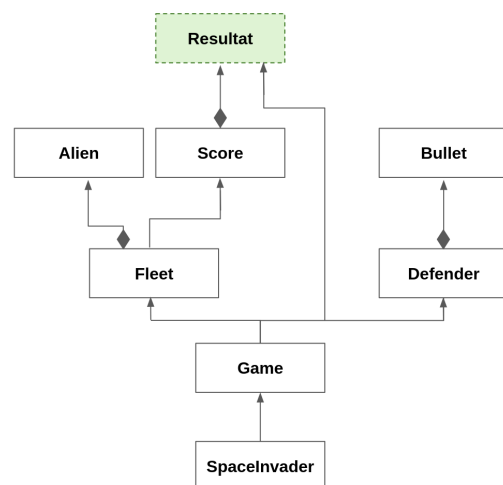
```
class Resultat(object):
    def __init__(self):
        self.lesScores=[]

    def ajout(self, score):
        self.lesScores.append(score)

    def __str__(self):
        chaine=str(self.lesScores[0])
        for e in self.lesScores[1:]:
            chaine=chaine+ "," + str(e)
        return chaine

    @classmethod
    def fromFile(cls, file):
        ...

    def toFile(self, file):
        ...
```



Cette classe gère une liste de scores, permettant de les ajouter, de les afficher sous forme de chaîne de caractères, et de les sauvegarder dans un fichier JSON (lesScores.json) avec toFile() et fromFile().

```
def __init__(self):
```

Dans cette méthode de définition d'instance, une liste vide self.lesScores est initialisée. Cette liste est censée contenir les scores des joueurs.

```
def ajout(self, score):
```

Cette méthode permet d'ajouter un score à la liste l self.lesScores de la classe Resultat.

```
def __str__(self):
```

Cette méthode spéciale permet de définir la représentation en chaîne de l'objet. Elle affiche le nom et le score de tous les joueurs de la liste self.lesScores de la manière suivante : Joueur1 : Score, Joueur2 : Score, Joueur3 : Score.

```
def fromFile(cls, file):
```

La méthode fromFile est une méthode de classe définie avec le décorateur @classmethod. Cette méthode est utilisée pour charger des données depuis un fichier JSON (lesScores.json) récupéré en paramètre file et les transformer en une instance de la classe Resultat, contenant une liste de scores.

```
def toFile(self, file):
```

Cette méthode permet de sauvegarder la liste des scores dans un fichier JSON (lesScores.json). Elle fonctionne en ouvrant le fichier, en chargeant les données existantes (si elles existent), puis en ajoutant les nouveaux scores à ces données avant de sauvegarder le tout dans le fichier (lesScores.json).

1.3 - Amélioration possible

Une amélioration possible serait de mieux gérer l'incrémentation du score. Actuellement, le score continue de s'incrémenter même après la fin du jeu, tant que le joueur n'a pas fermé la fenêtre de message (tkMessageBox) affichant la victoire ou la défaite du joueur. Cela fausse la mesure du temps exact que le joueur met pour gagner. En arrêtant l'incrémentation du score dès la fin du jeu et en enregistrant le temps écoulé, on pourrait obtenir une évaluation plus précise de la durée de la partie et du temps que chaque joueur prend pour gagner.

1.4 - Inclure une perspective IA

Il serait intéressant d'intégrer une dimension d'intelligence artificielle (IA) en utilisant l'algorithme Minimax pour ajuster dynamiquement la vitesse de déplacement des aliens, en contrôlant la variable self.alien_x_delta. Cette approche permettrait aux aliens d'adapter leur vitesse pour éviter d'être touchés par les projectiles du joueur. Parallèlement, l'IA pourrait également ajuster la vitesse des projectiles, en modifiant la variable self.speed, afin de réduire le risque de toucher un alien. Cette gestion adaptative de la vitesse, guidée par une stratégie basée sur l'algorithme Minimax, renforcerait le niveau de difficulté en permettant aux ennemis de réagir de manière plus intelligente aux actions du joueur. En ajustant

dynamiquement leur comportement en fonction des mouvements et des projectiles du joueur, les aliens deviendraient des adversaires plus imprévisibles, offrant ainsi une expérience de jeu plus immersive et stimulante.

Conclusion

Le jeu a été conçu de manière adéquate en respectant les consignes initiales. Cependant, une amélioration importante reste à apporter concernant la gestion du score du joueur pour afficher un score fidèle au temps que chaque joueur a mis pour gagner.